

tq-mtopt: tensor rank cross and matrix train cross representations for sample-efficient black-box optimization

Aleksandr Berezutskii¹, Alexander C. Paul¹, Michael Perelshtein¹, and Roman Ellerbrock¹

¹ Terra Quantum AG, Freddie-Mercury Str. 5, Munich, DE 80979

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 07 May 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

tq-mtopt is a lightweight Python library that implements several optimizers on discrete grids by extending matrix cross approximation to several types of tensor networks. The first, **Tensor Rank Cross (TRC)** is a cross approximation for tensor rank decomposition (also called canonical diadic decomposition (Candecomp)). The second, **Matrix Train Cross (MTC)** approximation is a cross approximation that combines features of TRC and Tensor Train (TT, also called Matrix Product State (MPS)) decomposition. Finally, the package also includes a general Tree Tensor Network (TTN) optimizer that can be applied to any user-defined tree structure. The methods build low-rank tensor representations of a function from a small number of function evaluations, then extract candidate optima directly from the representations. The package is designed specifically for high-dimensional functions with multiple local minima, where each function evaluation is computationally expensive.

Statement of need

Many real-world optimization problems are high-dimensional and non-convex, with expensive objective function evaluations. General-purpose gradient-free methods (for example, direct search or evolutionary strategies) are robust but often require many evaluations to locate good solutions. Tensor network decompositions offer an orthogonal strategy: they approximate the function on a discrete grid with a compact representation. In this structure, minima and maxima are obtained automatically as part of the compression (Dolgov & Savostyanov, 2025; Oseledets & Tyrtshnikov, 2010; Sozykin et al., 2022) and the number of function evaluations is tied to the chosen rank. However, different functions need different tensor-network architectures, so a single architecture is rarely optimal across different problems. The current package addresses this by providing optimizers for several architectures, such as tree tensor networks, tensor-rank decompositions, and hybrid matrix-train models combining tensor trains with tensor-rank structure. The primary use case is minimizing expensive black-box functions over high-dimensional discrete spaces, such as those arising in quantum chemistry, materials science, and hyperparameter optimization.

State of the field

The closest alternative tool is TTOpt (Sozykin et al., 2022), which applies maximum-volume cross approximation within a tensor train representation. It achieves strong performance on smooth, nearly unimodal functions. Dolgov and Savostyanov (Dolgov & Savostyanov, 2025) similarly apply tensor cross interpolation within a tensor train topology for discrete

combinatorial problems. General-purpose gradient-free methods such as SciPy's Differential Evolution and Dual Annealing are widely used but scale poorly with dimension, requiring evaluation budgets that grow exponentially in the number of variables.

tq-mtopt extends the space of available tensor-network architectures beyond the chain topology. The tensor rank decomposition represents the function as a sum of rank-1 terms and can capture correlation structures that a chain topology misses. The matrix train cross (MTC) format hybridizes tensor train and tensor rank, providing a richer family of approximations within the same sweep-based framework. The TTN optimizer generalizes this further to arbitrary tree topologies, enabling users to encode domain-specific variable dependencies directly into the network structure. Benchmarks on eleven standard test functions confirm that no single architecture dominates across all problem classes, which motivates providing all three in a unified package.

A new package was created rather than extending TTOpt because TRC and MTC require pivot-update rules that are fundamentally incompatible with TTOpt's sweep logic. Building from a clean design allowed a common skeleton abstraction, evaluation cache, and benchmarking harness to be shared across all three optimizers.

Functionality

- **Tensor Rank Cross (TRC) optimizer.** Computes a rank- r cross approximation of a tensor defined on a user-supplied product grid. Each sweep alternates one-leg updates (cross-approximation on one dimension while holding others fixed) with selection via maximum-volume principle or linear-sum-assignment on the cross matrix. It returns the updated rank- r skeleton (pivot set) and the current candidate minima/maxima.
- **Matrix Train Cross (MTC) optimizer.** Optimizes a function using cross approximation in an (N) -site matrix-train representation (a hybrid between tensor train and tensor rank decomposition) directly on the grid. The core step partitions dimensions into two groups and recombines them to form an $(r \times r)$ cross matrix. It then uses the Hungarian assignment to pick the next rank- r slice, followed by one-leg updates described above.
- **Tree Tensor Network (TTN) optimizer.** Performs cross approximation on a user-defined Tree Tensor Network for a given function. The library lets you specify the tree structure manually, or it can construct one automatically as either a nearly balanced tree or a chain (the tensor-train case). Each pass forms local Cartesian subgrids at internal nodes, evaluates the objective, builds edge-wise cross matrices, selects pivots (maximum-volume or Hungarian assignment), and applies one-leg updates. It stops when pivots stabilize or the evaluation budget is reached.
- **Hyperparameters, budgets, and logging.** Topology, ranks, sweeps, 1D grid parameters (uniform or custom), and seeds are user-controlled. Function evaluations are cached and recorded, and the framework tracks objective-call counts. Computational and evaluation cost are bounded by the chosen rank, number of sweeps, and grid size. Deterministic seeding enables fair common-random-numbers comparisons across methods.
- **Benchmarks & baselines.** The repo includes a benchmarking suite with CSV outputs and plots, comparing TRC and MTC against TTOpt and SciPy baselines (Differential Evolution and Dual Annealing). The benchmarks can conveniently be extended to other optimizers.

Software design

The central abstraction in **tq-mtopt** is the *skeleton*: a rank- r tensor of grid coordinate tuples representing the current low-rank approximation of the objective. All three optimizers share

85 this abstraction: each sweep constructs a local cross matrix from the current skeleton and
 86 selects new pivots from it. This shared design allows a single evaluation cache and optimization
 87 logger to be reused across TRC, MTC, and TTN without duplication.

88 Pivot selection is the primary algorithmic design choice. Two strategies are provided: the
 89 *maximum-volume* principle (Oseledets & Tyrtshnikov, 2010), which greedily selects the
 90 submatrix of largest absolute determinant and offers quasi-optimal approximation guarantees,
 91 and the *assignment* (Hungarian algorithm), which is better suited to cross matrices with
 92 structured permutation patterns. Both are supported for TRC and MTC while the TTN
 93 optimizer applies maximum-volume independently at each internal tree node.

94 The tree topology in the TTN optimizer is represented as a NetworkX directed graph, cleanly
 95 decoupling structure from optimization logic. Users may supply any valid tree or use built-in
 96 constructors for balanced trees or chains (recovering the tensor train special case). Evaluations
 97 are cached using the rounded coordinate tuple as a key, so repeated queries at pivot intersections
 98 — common in cross approximation — incur no additional objective calls.

99 Minimal example

```
import numpy as np
from tq_mtopt.grid import Grid, tensor_network_grid, build_node_grid
from tq_mtopt.network import balanced_tree, root
from tq_mtopt.optimization import (
    TensorRankOptimization,
    MatrixTrainOptimization,
    Objective,
    random_grid_points,
    tree_tensor_network_optimize,
)

# Black-box objective (accept **kwargs to ignore optimizer metadata like `epoch`)
def sphere(x, **_):
    x = np.asarray(x, dtype=float)
    return float(np.sum(x**2))

# 1) Primitive 1D grids
x0 = np.linspace(-2.0, 2.0, 51)
x1 = np.linspace(-2.0, 2.0, 51)
x2 = np.linspace(-2.0, 2.0, 51)
g0, g1, g2 = Grid(x0, coords=0), Grid(x1, coords=1), Grid(x2, coords=2)
primitives = [g0, g1, g2]

rank = 6
epochs = 8

# --- TRC ---
trc = TensorRankOptimization(primitives)
skel_trc = random_grid_points(primitives, n_samples=10, seed=42)
skel_trc = trc.optimize(skel_trc, function=sphere, num_epochs=epochs)
vals_trc = skel_trc.evaluate(sphere)
i_trc = int(np.argmin(vals_trc))
x_trc, f_trc = skel_trc.grid[i_trc], float(vals_trc[i_trc])
print("TRC -> x* =", np.round(x_trc, 4), "f* =", f"{f_trc:.6f}")

# --- MTC ---
```

```
mtc = MatrixTrainOptimization(primitives)
skel_mtc = random_grid_points(primitives, n_samples=10, seed=42)
skel_mtc = mtc.optimize(skel_mtc, function=sphere, num_epochs=epochs)
vals_mtc = skel_mtc.evaluate(sphere)
i_mtc = int(np.argmin(vals_mtc))
x_mtc, f_mtc = skel_mtc.grid[i_mtc], float(vals_mtc[i_mtc])
print("MTC -> x* =", np.round(x_mtc, 4), "f* =", f"{f_mtc:.6f}")

# --- TTN ---
G = balanced_tree(num_leaves=3, rank=rank, phys_dim=len(x0))
G = tensor_network_grid(G, primitive_grid=[x0, x1, x2])
obj = Objective(sphere)
G = tree_tensor_network_optimize(G, obj, num_sweeps=epochs)
df = obj.logger.dataframe
coord_cols = [c for c in df.columns if c.startswith("x")]
top_k = df.nsmallest(3, "f")[coord_cols + ["f"]].reset_index(drop=True)
top_k.index += 1
print("TTN top-3 optima:")
print(top_k.to_string())
```

Following the minimal example above, users can easily adapt the objective, discretization, rank schedule, and evaluation budget to their application. Additional usage examples as well as the detailed API documentation are provided in the [documentation](#).

Research impact statement

tq-mtopt ships with a reproducible benchmarking suite covering eleven global optimization test functions and a randomly parameterized multi-well potential—in both plain-grid and quantized tensor train (QTT) grid configurations across a range of ranks, sweep counts, and problem dimensions. All results are saved as CSV files and publication-quality plots, experiments are fully reproducible through fixed seeds. Full API documentation is hosted on ReadTheDocs with end-to-end usage examples for all three optimizers. The repository includes GitHub Actions pipelines for continuous integration: unit tests, linting, code security scanning, dependency review, and automated package releases to ensure long-term correctness and ease of adoption. The deterministic seeding infrastructure enables future head-to-head comparisons against new architectures without rerunning existing baseline experiments.

AI usage disclosure

Generative AI tools were used to assist with paper writing and documentation. The software implementation, algorithmic design, benchmarking, and core technical contributions were developed without AI assistance. All AI-assisted content has been reviewed and validated by the authors for technical accuracy and scholarly integrity.

References

- Dolgov, S., & Savostyanov, D. (2025). Tensor cross interpolation for global discrete optimization with application to bayesian network inference. *Computational Mathematics and Mathematical Physics*, 65(7), 1591–1604. <https://doi.org/10.1134/s0965542525700666>
- Oseledets, I., & Tyrtshnikov, E. (2010). TT-cross approximation for multidimensional arrays. *Linear Algebra and Its Applications*, 432(1), 70–88. <https://doi.org/10.1016/j.laa.2009.07.024>

- 127 Sozykin, K., Chertkov, A., Schutski, R., Phan, A.-H., Cichocki, A. S., & Oseledets, I.
128 (2022). TTOpt: A maximum volume quantized tensor train-based optimization and
129 its application to reinforcement learning. *Advances in Neural Information Processing*
130 *Systems*, 35, 26052–26065. <https://doi.org/10.52202/068431-1889>

DRAFT